

Fabricio Roskowski
Klaus Dieter Kupper

Avaliação 3 – Projeto RTL do Algoritmo de Máximo Divisor Comum

Itajaí – SC
Novembro de 2025

Sumário

1	INTRODUÇÃO	2
2	ESPECIFICAÇÃO DO SISTEMA	3
2.1	Resumo do Enunciado	3
2.2	Interface e Convenção de Sinais	3
3	METODOLOGIA DE PROJETO RTL	5
3.1	Algoritmo de Referência em C	5
3.2	Derivação da Máquina de Estados	6
3.3	Particionamento do Caminho de Dados	7
4	ARQUITETURA RTL PROPOSTA	8
4.1	Visão Geral	8
4.2	Bloco de Controle	8
4.3	Caminho de Dados	9
5	IMPLEMENTAÇÃO EM VHDL	10
5.1	Módulo Topo	10
5.2	Máquina de Controle	12
5.3	Caminho de Dados	14
5.4	Testbench	16
6	VALIDAÇÃO E SÍNTESE	20
6.1	Validação Funcional	20
6.2	Síntese e Análise RTL	20
7	CONCLUSÃO	23
	Referências	23
	REFERÊNCIAS	24

1 Introdução

Este relatório corresponde à Avaliação 3 da disciplina Projeto de Sistemas Digitais. A tarefa consiste em implementar em VHDL um circuito para calcular o Máximo Divisor Comum (MDC) de dois operandos de 8 bits, seguindo o procedimento trabalhado em aula: partir de uma função em C, derivar a máquina de estados, separar controle e caminho de dados, validar com ModelSim e sintetizar no Quartus Prime. Mantivemos o foco no essencial para a entrega: interface, FSM, códigos VHDL, simulações e métricas de síntese.

2 Especificação do Sistema

2.1 Resumo do Enunciado

O sistema deve receber duas entradas de dados (**i_x** e **i_y**), cada uma com 8 bits, e produzir em **o_d** o MDC entre esses operandos. O fluxo é controlado por uma entrada de comando **i_go** e uma saída de status **o_rdy**. Enquanto **o_rdy = 1**, o circuito está pronto para receber um novo par de operandos; quando **i_go** é ativado, o circuito captura os dados, executa o algoritmo e devolve **o_rdy = 1** apenas após o resultado ser disponibilizado em **o_d**. A implementação deve ser hierárquica, com módulos separados para topo, controle e caminho de dados, seguindo a convenção de nomes indicada no enunciado (Tabela 1).

2.2 Interface e Convenção de Sinais

A Tabela 1 apresenta um resumo das interfaces externas, enquanto a Tabela 2 registra os principais sinais internos que emergiram do processo de particionamento.

Tabela 1 – Interface externa do sistema **mdc_top**.

Identificador	Tipo	Descrição
i_CLK	Entrada (1 bit)	Clock principal do sistema.
i_RSTn	Entrada (1 bit)	Reset assíncrono ativo em nível baixo.
i_GO	Entrada (1 bit)	Pulso que inicia o processamento.
i_X	Entrada (8 bits)	Operando A sem sinal.
i_Y	Entrada (8 bits)	Operando B sem sinal.
o_D	Saída (8 bits)	Resultado do MDC após a convergência.
o_RDY	Saída (1 bit)	Indica se o módulo está pronto para nova transação.

Tabela 2 – Sinais internos definidos no particionamento controle/caminho de dados.

Sinal	Origem/Destino	Função
w_Rx_ld, w_Ry_ld	Controle → Data-path	Habilitam a carga inicial dos registradores r_X e r_Y .
w_Rx_clr, w_Ry_clr	Controle → Data-path	Forçam a limpeza assíncrona dos registradores (após reset e em condições excepcionais).
w_Rx_sub_ld, w_Ry_sub_ld	Controle → Data-path	Atualizam r_X ou r_Y com o resultado da subtração apropriada.
w_X_eq_Y	Datapath → Controle	Indica igualdade entre r_X e r_Y .
w_X_lt_Y	Datapath → Controle	Indica que r_X é estritamente menor que r_Y .
r_X, r_Y	Datapath	Registradores de armazenamento intermediário do algoritmo.

3 Metodologia de Projeto RTL

3.1 Algoritmo de Referência em C

O algoritmo de MDC utilizado como ponto de partida segue o método de subtrações sucessivas, incluindo a lógica de handshake com os sinais `i_go` e `o_rdy`. O código completo é apresentado na Listagem 3.1.

Listing 3.1 – Função de referência em C para o cálculo do MDC com handshake

```
1 #include <stdint.h>
2 #include <stdbool.h>
3
4 static bool i_go = false;
5 static uint8_t i_x = 0U;
6 static uint8_t i_y = 0U;
7
8 static uint8_t gcd_subtractive(uint8_t x, uint8_t y)
9 {
10     while (x != y) {
11         if (x < y) {
12             y -= x;
13         } else {
14             x -= y;
15         }
16     }
17     return x;
18 }
19
20 int main(void)
21 {
22     uint8_t o_d = 0U;
23     bool o_rdy = true;
24
25     while (true) {
26         while (!i_go) {
27             /* idle */
28         }
29
30         o_rdy = false;
31         o_d = gcd_subtractive(i_x, i_y);
```

```

32   o_rdy = true;
33   }
34 }

```

3.2 Derivação da Máquina de Estados

A partir do algoritmo, identificaram-se seis estados para o controle: `s_IDLE`, `s_LOAD`, `s_COMPARE`, `s_SUB_X`, `s_SUB_Y` e `s_DONE`. A Figura 1 mostra o diagrama desenhado manualmente (`media/states.jpeg`); na Seção de síntese, a Figura 4 confirma essa mesma FSM exportada pelo Quartus Prime. A Tabela 3 detalha as transições e saídas relevantes.

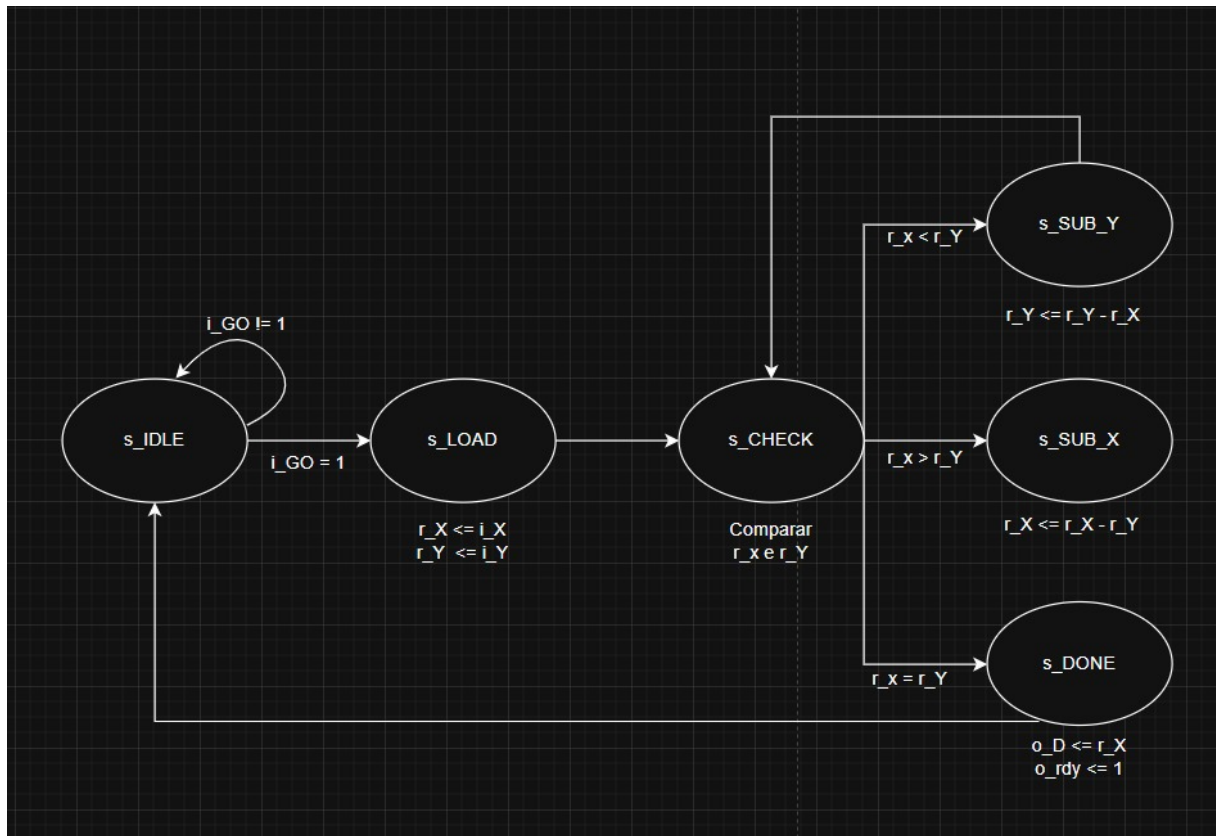


Figura 1 – Diagrama inicial da máquina de estados elaborado manualmente.

Tabela 3 – Tabela de transições da FSM de controle.

Estado	Ação principal	Condição	Próximo estado
s_IDLE	$o_RDY = 1$	$i_GO = 1$ caso contrário	s_LOAD s_IDLE
s_LOAD	Carrega r_X e r_Y	–	s_COMPARE
s_COMPARE	Avalia r_X e r_Y	$w_X_EQ_Y = 1$ $w_X_LT_Y = 1$ demais casos	s_DONE s_SUB_Y s_SUB_X s_COMPARE
s_SUB_X	Atualiza r_X com $r_X - r_Y$	–	s_COMPARE
s_SUB_Y	Atualiza r_Y com $r_Y - r_X$	–	s_COMPARE
s_DONE	$o_RDY = 1$, mantém o_D	$i_GO = 0$ $i_GO = 1$	s_IDLE s_DONE

3.3 Particionamento do Caminho de Dados

O caminho de dados contém dois registradores (r_X e r_Y) e dois blocos de subtração sempre ativos. Os resultados intermediários ($w_sub_x = r_X - r_Y$ e $w_sub_y = r_Y - r_X$) alimentam os sinais de atualização condicional controlados pela FSM. O bloco de controle decide qual registrador deve receber uma nova subtração por meio das linhas $w_Rx_sub_ld$ e $w_Ry_sub_ld$, evitando underflow ao nunca habilitar a operação inválida. As comparações de igualdade ($w_X_eq_Y$) e menor ($w_X_lt_Y$) são calculadas continuamente e retroalimentam o controle.

4 Arquitetura RTL Proposta

4.1 Visão Geral

A Figura 2 apresenta o diagrama RTL em alto nível, destacando a interação entre o bloco de controle (`mdc_control`) e o caminho de dados (`mdc_datapath`). As setas são etiquetadas com os sinais definidos na Tabela 2.

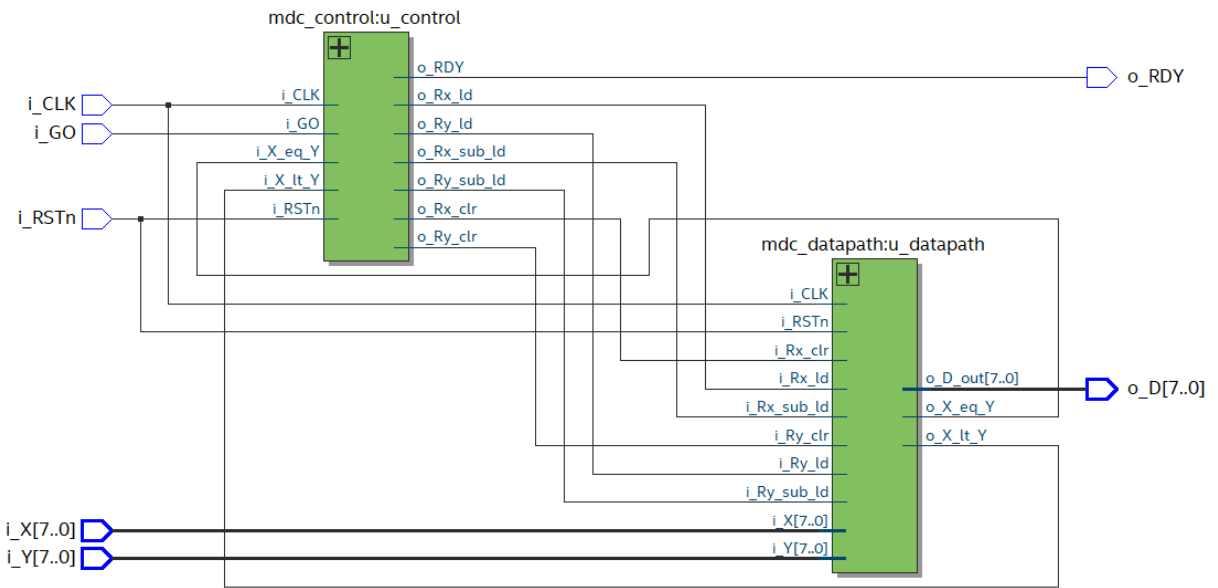


Figura 2 – Particionamento em controle e caminho de dados do sistema MDC.

4.2 Bloco de Controle

O bloco `mdc_control` é responsável por sequenciar o algoritmo, garantindo que apenas operações válidas de subtração sejam realizadas. O uso de estados explícitos facilita a inclusão de novas condições de parada (por exemplo, detecção de operandos nulos) sem alterar o caminho de dados. As saídas `w_Rx_ld`, `w_Ry_ld`, `w_Rx_clr`, `w_Ry_clr`, `w_Rx_sub_ld` e `w_Ry_sub_ld` são exclusivamente geradas pelo controle e utilizadas como sinais de habilitação dentro do datapath. O sinal `o_RDY` permanece em nível alto nos estados `s_IDLE` e `s_DONE`, assegurando o handshake requisitado pelo enunciado (`o_RDY = 1` sempre que o módulo estiver apto a receber nova transação).

4.3 Caminho de Dados

O caminho de dados mantém os operandos em registradores e atualiza seus valores com base na decisão do controle. As subtrações são realizadas em paralelo, permitindo que o controlador selecione o resultado correto com atraso mínimo. O resultado final é exposto diretamente em `o_D`, que é repassado ao módulo topo.

5 Implementação em VHDL

Os códigos foram escritos seguindo o guia de estilo da avaliação: palavras reservadas em minúsculas, sinais em maiúsculas com prefixos (*i_*, *o_*, *w_*, *r_*), tabulação de dois espaços e comentários em inglês para evitar problemas de codificação.

5.1 Módulo Topo

A Listagem 5.1 mostra o módulo topo `mdc_top`, responsável por conectar os blocos de controle e caminho de dados e disponibilizar a interface externa.

Listing 5.1 – Módulo topo do sistema MDC

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mdc_top is
6    port (
7      i_CLK : in std_logic;
8      i_RSTn : in std_logic;
9      i_GO   : in std_logic;
10     i_X    : in std_logic_vector(7 downto 0);
11     i_Y    : in std_logic_vector(7 downto 0);
12     o_D    : out std_logic_vector(7 downto 0);
13     o_RDY  : out std_logic
14   );
15 end entity mdc_top;
16
17 architecture rtl of mdc_top is
18
19   signal w_Rx_ld    : std_logic;
20   signal w_Ry_ld    : std_logic;
21   signal w_Rx_clr   : std_logic;
22   signal w_Ry_clr   : std_logic;
23   signal w_Rx_sub_ld : std_logic;
24   signal w_Ry_sub_ld : std_logic;
25
26   signal w_X_eq_Y   : std_logic;
27   signal w_X_lt_Y   : std_logic;
28   signal w_D_out    : std_logic_vector(7 downto 0);

```

```
29
30 begin
31
32   u_datapath : entity work.mdc_datapath
33     port map (
34       i_CLK      => i_CLK,
35       i_RSTn     => i_RSTn,
36       i_X        => i_X,
37       i_Y        => i_Y,
38       i_Rx_ld    => w_Rx_ld,
39       i_Ry_ld    => w_Ry_ld,
40       i_Rx_clr   => w_Rx_clr,
41       i_Ry_clr   => w_Ry_clr,
42       i_Rx_sub_ld => w_Rx_sub_ld,
43       i_Ry_sub_ld => w_Ry_sub_ld,
44       o_X_eq_Y   => w_X_eq_Y,
45       o_X_lt_Y   => w_X_lt_Y,
46       o_D_out    => w_D_out
47     );
48
49   u_control : entity work.mdc_control
50     port map (
51       i_CLK      => i_CLK,
52       i_RSTn     => i_RSTn,
53       i_GO       => i_GO,
54       i_X_eq_Y   => w_X_eq_Y,
55       i_X_lt_Y   => w_X_lt_Y,
56       o_Rx_ld    => w_Rx_ld,
57       o_Ry_ld    => w_Ry_ld,
58       o_Rx_clr   => w_Rx_clr,
59       o_Ry_clr   => w_Ry_clr,
60       o_Rx_sub_ld => w_Rx_sub_ld,
61       o_Ry_sub_ld => w_Ry_sub_ld,
62       o_RDY      => o_RDY
63     );
64
65   o_D <= w_D_out;
66
67 end architecture rtl;
```

5.2 Máquina de Controle

A Listagem 5.2 apresenta a implementação da FSM derivada na Tabela 3. Os estados são codificados em um tipo enumerado `t_STATE`, e os sinais de controle seguem diretamente a tabela de transições.

Listing 5.2 – Implementação da FSM de controle

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mdc_control is
6      port (
7          i_CLK      : in std_logic;
8          i_RSTn     : in std_logic;
9          i_GO       : in std_logic;
10         i_X_eq_Y    : in std_logic;
11         i_X_lt_Y    : in std_logic;
12         o_Rx_ld     : out std_logic;
13         o_Ry_ld     : out std_logic;
14         o_Rx_clr    : out std_logic;
15         o_Ry_clr    : out std_logic;
16         o_Rx_sub_ld : out std_logic;
17         o_Ry_sub_ld : out std_logic;
18         o_RDY       : out std_logic
19     );
20 end entity mdc_control;
21
22 architecture rtl of mdc_control is
23
24     type t_state is (s_IDLE, s_LOAD, s_CHECK, s_SUB_Y, s_SUB_X, s_DONE);
25     signal s_CUR : t_state := s_IDLE;
26     signal s_NXT : t_state;
27
28     signal v_Rx_ld, v_Ry_ld, v_Rx_clr, v_Ry_clr, v_Rx_sub_ld, v_Ry_sub_ld, v_RDY
29         : std_logic;
30
31 begin
32     p_state : process (i_RSTn, i_CLK)
33     begin
34         if i_RSTn = '0' then

```

```

35     s_CUR <= s_IDLE;
36     elsif rising_edge(i_CLK) then
37         s_CUR <= s_NXT;
38     end if;
39 end process p_state;
40
41 p_comb : process (s_CUR, i_GO, i_X_eq_Y, i_X_lt_Y)
42 begin
43     v_Rx_ld    <= '0';
44     v_Ry_ld    <= '0';
45     v_Rx_clr   <= '0';
46     v_Ry_clr   <= '0';
47     v_Rx_sub_ld <= '0';
48     v_Ry_sub_ld <= '0';
49     v_RDY      <= '0';
50     s_NXT      <= s_CUR;
51
52     case s_CUR is
53         when s_IDLE =>
54             v_RDY <= '1';
55             if i_GO = '1' then
56                 s_NXT <= s_LOAD;
57             else
58                 s_NXT <= s_IDLE;
59             end if;
60
61         when s_LOAD =>
62             v_Rx_ld <= '1';
63             v_Ry_ld <= '1';
64             s_NXT <= s_CHECK;
65
66         when s_CHECK =>
67             if i_X_eq_Y = '1' then
68                 s_NXT <= s_DONE;
69             else
70                 if i_X_lt_Y = '1' then
71                     s_NXT <= s_SUB_Y;
72                 else
73                     s_NXT <= s_SUB_X;
74                 end if;
75             end if;
76

```

```

77     when s_SUB_Y =>
78         v_Ry_sub_ld <= '1';
79         s_NXT <= s_CHECK;
80
81     when s_SUB_X =>
82         v_Rx_sub_ld <= '1';
83         s_NXT <= s_CHECK;
84
85     when s_DONE =>
86         v_RDY <= '1';
87         if i_GO = '0' then
88             s_NXT <= s_IDLE;
89         else
90             s_NXT <= s_DONE;
91         end if;
92
93     when others =>
94         s_NXT <= s_IDLE;
95     end case;
96 end process p_comb;
97
98 o_Rx_ld    <= v_Rx_ld;
99 o_Ry_ld    <= v_Ry_ld;
100 o_Rx_clr   <= v_Rx_clr;
101 o_Ry_clr   <= v_Ry_clr;
102 o_Rx_sub_ld <= v_Rx_sub_ld;
103 o_Ry_sub_ld <= v_Ry_sub_ld;
104 o_RDY      <= v_RDY;
105
106 end architecture rtl;

```

5.3 Caminho de Dados

A Listagem 5.3 contém a descrição do caminho de dados. Os cálculos são realizados com a biblioteca `numeric_std`, garantindo aritmética sem sinal consistente com a especificação.

Listing 5.3 – Descrição do caminho de dados

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;

```

```

4
5 entity mdc_datapath is
6   port (
7     i_CLK      : in  std_logic;
8     i_RSTn     : in  std_logic;
9     i_X        : in  std_logic_vector(7 downto 0);
10    i_Y        : in  std_logic_vector(7 downto 0);
11    i_Rx_ld     : in  std_logic;
12    i_Ry_ld     : in  std_logic;
13    i_Rx_clr    : in  std_logic;
14    i_Ry_clr    : in  std_logic;
15    i_Rx_sub_ld : in  std_logic;
16    i_Ry_sub_ld : in  std_logic;
17    o_X_eq_Y    : out std_logic;
18    o_X_lt_Y    : out std_logic;
19    o_D_out     : out std_logic_vector(7 downto 0)
20  );
21 end entity mdc_datapath;
22
23 architecture rtl of mdc_datapath is
24
25   signal r_X : std_logic_vector(7 downto 0);
26   signal r_Y : std_logic_vector(7 downto 0);
27
28   signal w_sub_x : unsigned(7 downto 0);
29   signal w_sub_y : unsigned(7 downto 0);
30
31 begin
32
33   w_sub_x <= unsigned(r_X) - unsigned(r_Y);
34   w_sub_y <= unsigned(r_Y) - unsigned(r_X);
35
36   o_X_eq_Y <= '1' when r_X = r_Y else '0';
37   o_X_lt_Y <= '1' when unsigned(r_X) < unsigned(r_Y) else '0';
38
39   o_D_out <= r_X;
40
41   p_reg_x : process (i_RSTn, i_CLK)
42   begin
43     if i_RSTn = '0' then
44       r_X <= (others => '0');
45     elsif rising_edge(i_CLK) then

```



```

46     if i_Rx_clr = '1' then
47         r_X <= (others => '0');
48     elsif i_Rx_ld = '1' then
49         r_X <= i_X;
50     elsif i_Rx_sub_ld = '1' then
51         r_X <= std_logic_vector(w_sub_x);
52     end if;
53 end if;
54 end process p_reg_x;
55
56 p_reg_y : process (i_RSTn, i_CLK)
57 begin
58     if i_RSTn = '0' then
59         r_Y <= (others => '0');
60     elsif rising_edge(i_CLK) then
61         if i_Ry_clr = '1' then
62             r_Y <= (others => '0');
63         elsif i_Ry_ld = '1' then
64             r_Y <= i_Y;
65         elsif i_Ry_sub_ld = '1' then
66             r_Y <= std_logic_vector(w_sub_y);
67         end if;
68     end if;
69 end process p_reg_y;
70
71 end architecture rtl;

```

5.4 Testbench

O testbench da Listagem 5.4 instancia o módulo topo, gera o clock, aplica vetores de teste e aguarda o término de cada transação antes de iniciar a próxima. As assertivas de verificação podem ser ampliadas conforme necessário.

Listing 5.4 – Testbench funcional para o sistema MDC

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity tb_mdc is
6 end entity;
7

```

```
8 architecture sim of tb_mdc is
9
10     signal i_CLK : std_logic := '0';
11     signal i_RSTn : std_logic := '0';
12     signal i_GO  : std_logic := '0';
13     signal i_X   : std_logic_vector(7 downto 0) := (others => '0');
14     signal i_Y   : std_logic_vector(7 downto 0) := (others => '0');
15     signal o_D   : std_logic_vector(7 downto 0);
16     signal o_RDY : std_logic;
17
18     constant CLK_PERIOD : time := 10 ns;
19
20 begin
21
22     u_dut : entity work.mdc_top
23         port map (
24             i_CLK => i_CLK,
25             i_RSTn => i_RSTn,
26             i_GO  => i_GO,
27             i_X   => i_X,
28             i_Y   => i_Y,
29             o_D   => o_D,
30             o_RDY => o_RDY
31         );
32
33     p_clk : process
34     begin
35         while true loop
36             i_CLK <= '0';
37             wait for CLK_PERIOD/2;
38             i_CLK <= '1';
39             wait for CLK_PERIOD/2;
40         end loop;
41     end process p_clk;
42
43     p_stim : process
44     begin
45         i_RSTn <= '0';
46         wait for CLK_PERIOD;
47         i_RSTn <= '1';
48
49         i_X <= std_logic_vector(to_unsigned(14,8));
```

```
50     i_Y <= std_logic_vector(to_unsigned(21,8));
51     wait for CLK_PERIOD;
52     i_GO <= '1';
53     wait for CLK_PERIOD;
54     i_GO <= '0';
55
56     wait until o_RDY = '1';
57     assert o_D = std_logic_vector(to_unsigned(7,8))
58         report "Test1 FAILED: gcd(14,21) != 7" severity error;
59
60     wait for 30 ns;
61
62     i_X <= std_logic_vector(to_unsigned(100,8));
63     i_Y <= std_logic_vector(to_unsigned(25,8));
64     wait for 20 ns;
65     i_GO <= '1';
66     wait for 10 ns;
67     i_GO <= '0';
68
69     wait until o_RDY = '1';
70     assert o_D = std_logic_vector(to_unsigned(25,8))
71         report "Test2 FAILED: gcd(100,25) != 25" severity error;
72
73     wait for 50 ns;
74
75     i_X <= std_logic_vector(to_unsigned(42,8));
76     i_Y <= std_logic_vector(to_unsigned(42,8));
77     wait for 20 ns;
78     i_GO <= '1';
79     wait for 10 ns;
80     i_GO <= '0';
81     wait until o_RDY = '1';
82     assert o_D = std_logic_vector(to_unsigned(42,8))
83         report "Test3 FAILED: gcd(42,42) != 42" severity error;
84
85     wait for 50 ns;
86
87     report "SIMULATION COMPLETE" severity note;
88     wait;
89 end process p_stim;
90
91 end architecture sim;
```



6 Validação e Síntese

6.1 Validação Funcional

A verificação comportamental foi realizada no ModelSim/Questa com o *testbench* da Seção anterior. O script `simulate.tcl` compila os arquivos VHDL no modo 2008, executa a simulação automática e abre a visualização das ondas. As formas capturadas na Figura 3 destacam os pulsos de `i_GO`, a evolução de `r_X/r_Y` e o retorno de `o_RDY` apenas após o resultado estabilizado. Nenhuma assertiva foi violada durante os três vetores exercitados (14, 21, 100, 25 e 42, 42).

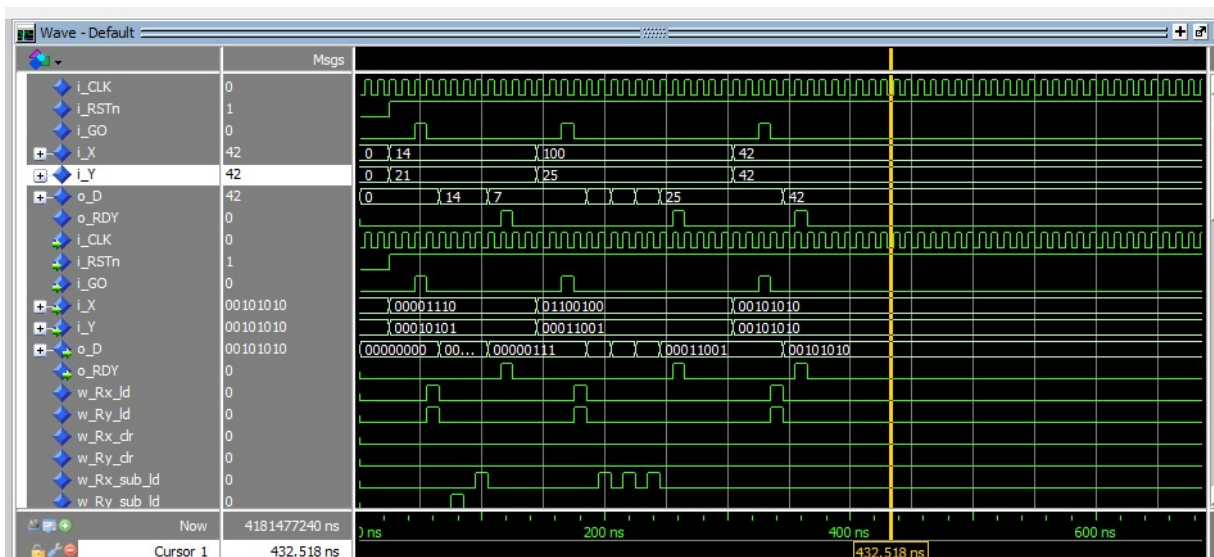


Figura 3 – Espaço reservado para as formas de onda obtidas no ModelSim.

As formas de onda devem evidenciar: (i) o pulso de `i_GO` na captura dos operandos; (ii) a evolução dos registradores `r_X` e `r_Y` convergindo para o resultado; (iii) a retomada de `o_RDY` apenas quando `o_D` está estabilizado. Na execução mais recente do *testbench*, cada transação terminou sem erros de *assert*, confirmando que os três vetores exercitam tanto as ramificações `s_SUB_X` quanto `s_SUB_Y`.

6.2 Síntese e Análise RTL

A síntese foi realizada no Quartus Prime Standard utilizando a hierarquia apresentada na Figura 2. Após a compilação do projeto com a entidade `mdc_top`, foram exportados o diagrama RTL (Figuras 2 e 5) e as métricas de utilização de recursos incluídas na Tabela 4. A análise de temporização considerou o modelo *Slow 1,10 V 85°C*, cujo relatório indica Fmax de 381,24 MHz para o único domínio de clock.

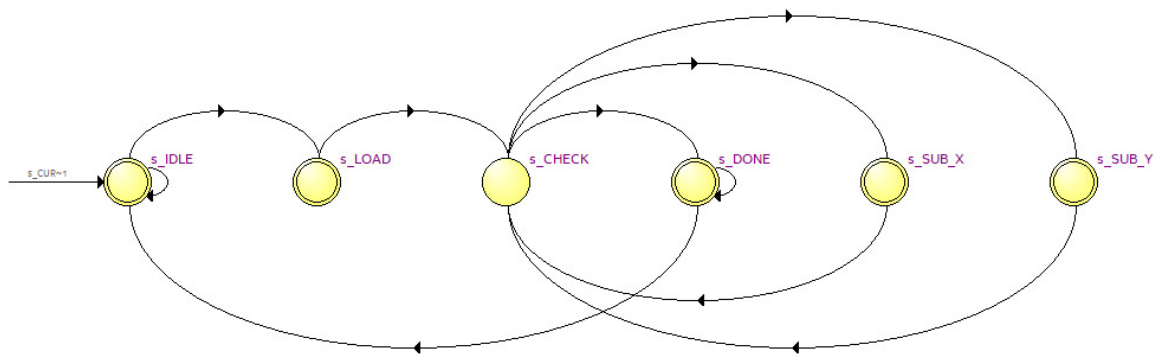


Figura 4 – FSM sintetizada pelo Quartus Prime, confirmando a implementação VHDL.

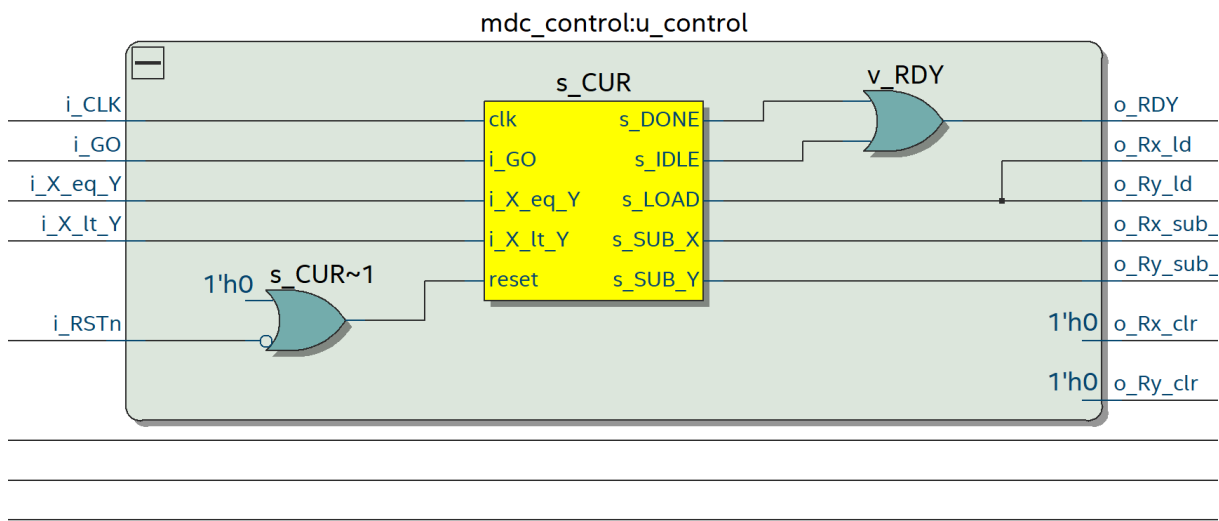


Figura 5 – Visão RTL do bloco de controle gerada pelo Quartus Prime.

Tabela 4 – Template para registrar os custos e o desempenho após a síntese.

Métrica	Valor	Observações
Número de ALMs	18	Dispositivo reporta recursos em ALMs (Cyclone V).
Número de LUTs	34	Combinational ALUTs (Fitter Report).
Número de FFs	22	Dedicated Logic Registers.
Frequência máxima (Fmax)	381,24 MHz	Timing Analyzer – Slow 1,10 V 85°C.

Os números de síntese mostram que a solução ocupa 18 ALMs do Cyclone V, com 34 ALUTs efetivamente utilizados para as operações de subtração/comparação e 22 registradores dedicados aos elementos de estado. A Fmax de 381,24 MHz indica ampla margem em relação à frequência-alvo típica (50 MHz), confirmando que a lógica combinacional entre relógios é leve. O diagrama RTL da Figura 5 evidencia que o Quartus inferiu corretamente a máquina de estados enumerada e multiplexadores controlados, sem inserção de blocos extras inesperados.

7 Conclusão

O projeto do circuito de Máximo Divisor Comum foi concluído com sucesso, atendendo aos requisitos da Avaliação 3. A abordagem RTL separou claramente controle e caminho de dados, permitindo verificar o algoritmo de subtrações sucessivas no ModelSim e sintetizá-lo no Quartus Prime. As formas de onda comprovam o handshake `i_GO/o_RDY` e a convergência dos registradores `r_X/r_Y`, enquanto o relatório de síntese mostra baixo consumo de recursos (18 ALMs) e elevada margem temporal (Fmax de 381,24 MHz). Esses resultados evidenciam que a implementação está pronta para integração em sistemas maiores ou para migração a dispositivos FPGA reais.

Referências

- [1] Xilinx. *VHDL Coding Style Guidelines*. AMD Adaptive Computing Documentation, 2023.
- [2] Intel. *Quartus Prime Standard Edition User Guide*. 2024.
- [3] Siemens EDA. *ModelSim Simulation User Manual*. 2024.